Comparing At-Most-One Encodings for SAT-Based Sudoku Solving

An empirical study from 9×9 to 36×36 grids, with ILP and CP baselines

Dang Cong Anh

Student ID: 24021357

Abstract

I study how the choice of at-most-one (AMO) encoding affects SAT-based Sudoku solving. A Sudoku of side $n = b^2$ is encoded with n^3 Boolean variables and $4n^2$ groups of n literals, each carrying an at-least-one (ALO) and an AMO constraint. I compare eight AMO encodings (pairwise, sequential counter, commander, binary, product, and PySAT's seqcounter, bitwise and ladder), give closed-form clause and variable counts for them, and check these formulas against the CNFs that the implementation generates for grids up to 64×64. On 9×9 to 36×36 puzzles (three punctured puzzles per size, a 60 s limit) I run each encoding with four PySAT solvers (CaDiCaL, Glucose, Kissat, MiniSat), and compare with CP-SAT, Gurobi, CPLEX (MIP and CP) and CBC. The CNF sizes differ substantially, for example the pairwise encoding of a 49×49 grid has about 8.5 times as many clauses as the product encoding. The run-time picture is less clear: for $n \leq 25$ all medians are below 0.52 s, and for $n = 36$ the best encoding depends on the solver, so I do not identify an overall fastest encoder. Removing the logically redundant per-unit ALO clauses slowed CaDiCaL down at the larger sizes. The free editions of Gurobi and CPLEX refuse models from 16×16 onwards, so those baselines are reported as unavailable rather than as timings. All code, raw results and puzzles are public.

1 Introduction

Sudoku is a convenient testbed for propositional encodings: the rules are short, instances scale by changing a single parameter, and every solution can be checked independently. A Sudoku of side $n = b^2$ (box size b) has n^2 cells, and each of its $4n^2$ constraint groups—one per cell, row–value, column–value and box–value pair—requires that exactly one of n candidates is chosen. In CNF the "at least one" half of such a constraint is a single clause, whereas the "at most one" half can be written in many ways that trade the number of clauses against the number of auxiliary variables. Because every Sudoku constraint is of this kind, the choice of AMO encoding fixes the size and shape of the whole formula.

This report examines that choice empirically. I do not propose new encodings; the encodings are taken from the literature and implemented (or called through PySAT) in a common framework. My contributions are:

- a complete variable and constraint model of Sudoku in CNF, with an explicit ALO/AMO formulation and closed-form (or recursive) clause and variable counts for eight AMO encodings (Section 3), which I verified against the CNFs generated by the implementation for all 47 measured encoding sizes;
- an evaluation on 9×9 to 36×36 puzzles with four SAT solvers, an ablation of the redundant per-unit ALO clauses, a comparison with CP-SAT, Gurobi, CPLEX and CBC, and an encode-only experiment up to 64×64 (Section 6);

- a reproducible harness that records the environment, content hashes of the puzzles and an independent validity check for each solution, together with an audit script (Section 4).

I report what the data support. With three puzzles per size and one run per configuration, differences between encodings are descriptive and are not tested for statistical significance (Section 7).

The remainder of the report is organised as follows. Section 2 defines the problem and reviews related work. Section 3 presents the encoding. Sections 4 and 5 describe the implementation and the experimental setup, Section 6 reports and discusses the results, and Sections 7 and 8 state the limitations and conclude.

2 Problem and background

2.1 Sudoku

Let $b \geq 1$ be the box size and $n = b^2$ the side of the grid. A *Sudoku grid* is a map $g : \{0, \ldots, n-1\}^2 \to \{0, \ldots, n-1\}$ in which every row, every column and every one of the n boxes ($b \times b$ sub-squares) contains each value exactly once. A *puzzle* is a set of *clues* (r, c, v), and a *solution* is a grid with $g(r, c) = v$ for every clue. The task is to find a solution or to report that none exists. Deciding whether a generalised Sudoku has a solution is NP-complete [1], so no polynomial-time algorithm is expected and SAT solvers are a natural tool.

2.2 SAT and CNF

A SAT instance is a propositional formula in conjunctive normal form (CNF): a conjunction of clauses, each a disjunction of literals. Modern conflict-driven clause-learning (CDCL) solvers such as MiniSat [2], Glucose [3], CaDiCaL and Kissat [4] decide such formulas, and PySAT [5] gives uniform access to them from Python. For a list of literals $L = (\ell_1, \ldots, \ell_m)$ I write

$$\mathrm{ALO}(L) = \bigvee_{i=1}^{m} \ell_i, \tag{1}$$

$$\mathrm{AMO}(L) \equiv \bigwedge_{1 \leq i < j \leq m} (\neg \ell_i \vee \neg \ell_j). \tag{2}$$

$\mathrm{ALO}(L)$ is already one clause. Equation (2) defines the meaning of $\mathrm{AMO}(L)$ and is also its *pairwise* CNF; other encodings produce different clause sets, usually over extra auxiliary variables, that are equivalent to (2) when the auxiliary variables are projected away.

2.3 Related work

Sudoku as a SAT problem was studied by Lynce and Ouaknine [6], who compare a minimal and an extended encoding, and by Weber [7]. The AMO constraint has many encodings: the sequential counter [8], the commander encoding [9], the binary (bitwise) encoding [10], the product encoding [11] and the ladder encoding [12]. Their trade-offs between clauses, auxiliary variables and propagation strength are well known in general terms; here I measure them on one concrete, regular problem family and report sizes and times side by side for eight variants, including the three that PySAT generates itself. As baselines I use the CP-SAT solver of OR-Tools [13], the MIP solvers Gurobi [14] and CPLEX [15] (and its CP Optimizer), and CBC [16] through PuLP [17].

3 SAT encoding

3.1 Variables

Indices r, c, v range over $\{0, \dots, n-1\}$. The Boolean variable $x_{r,c,v}$ is true iff cell (r, c) holds value v. The n^3 variables are numbered $\mathrm{var}(r, c, v) = rn^2 + cn + v + 1$, and auxiliary variables introduced by an AMO encoding are numbered after them.

3.2 Constraints

Let $\mathrm{Box}(\beta)$ denote the cells of box $\beta \in \{0, \dots, n-1\}$. The CNF is the conjunction of:

$$\mathrm{ALO}(x_{r,c,0}, \dots, x_{r,c,n-1}) \ \wedge \ \mathrm{AMO}(x_{r,c,0}, \dots, x_{r,c,n-1}) \qquad \forall r, c \quad \text{(cell)} \tag{3}$$

$$\mathrm{ALO}(x_{r,0,v}, \dots, x_{r,n-1,v}) \ \wedge \ \mathrm{AMO}(x_{r,0,v}, \dots, x_{r,n-1,v}) \qquad \forall r, v \quad \text{(row)} \tag{4}$$

$$\mathrm{ALO}(x_{0,c,v}, \dots, x_{n-1,c,v}) \ \wedge \ \mathrm{AMO}(x_{0,c,v}, \dots, x_{n-1,c,v}) \qquad \forall c, v \quad \text{(column)} \tag{5}$$

$$\mathrm{ALO}((x_{r,c,v})_{(r,c)\in\mathrm{Box}(\beta)}) \ \wedge \ \mathrm{AMO}((x_{r,c,v})_{(r,c)\in\mathrm{Box}(\beta)}) \qquad \forall \beta, v \quad \text{(box)} \tag{6}$$

$$x_{r,c,v} \qquad \forall \text{ clues } (r, c, v) \tag{7}$$

Constraints (3)–(6) form $n^2 + 3n^2 = 4n^2$ groups of exactly n literals each, which is why the size of the whole formula is governed by the cost $A(n)$ (clauses) and $V(n)$ (auxiliary variables) of a single AMO over n literals. Every distinct clue adds one unit clause (duplicates of the same clue are merged, clues that disagree on a cell are rejected before encoding). If c is the number of distinct clues, then

$$\#\text{clauses} = 4n^2\big(1 + A(n)\big) + c, \qquad \#\text{variables} = n^3 + 4n^2\,V(n). \tag{8}$$

Proposition 1. *The ALO clauses of (4)–(6) are implied by (3) together with the AMO parts of (4)–(6).*

Proof. Take a unit (row, column or box) with its n cells. By (3) every cell holds exactly one value. The AMO for value v in the unit says no two cells of the unit hold v, so the n cells hold n pairwise different values out of n possible values. Every value therefore occurs, which is the ALO clause for that unit and value. □

Such redundant clauses can still change solver behaviour, so I measure their effect rather than assuming it is neutral (ablation, Section 6). The encoder flag `unit_alo` (command line `--no-unit-alo`) removes them and keeps the per-cell ALO; the clause count then drops by exactly $3n^2$, and (8) becomes $4n^2A(n) + n^2 + c$.

Proposition 2. *For each AMO encoding used here, the models of the CNF projected onto the variables $x_{r,c,v}$ are exactly the solutions of the puzzle (encoded as grids), provided that the AMO encoding is equivalent to (2) after projecting away its auxiliary variables.*

Proof. A solution satisfies every group of (3)–(6) with exactly one true literal, so each AMO is satisfied (every encoding admits an auxiliary assignment when at most one literal is true) and so is each ALO. Conversely, a model has exactly one value per cell by (3), and no value repeated in a row, column or box by the AMO constraints; with Proposition 1 all unit constraints hold, hence it is a Sudoku grid, and the clue units force agreement with the puzzle. □

The hypothesis of Proposition 2 is verified exhaustively for every encoder in the repository's test suite (all single-literal and no-literal assignments are satisfiable and every pair of literals is unsatisfiable, for group sizes 2 to 30, 36 and 50). Different auxiliary assignments may represent the same grid for some encodings, so uniqueness checking blocks only the n^3 cell variables (Section 4).

Table 1: Clauses $A(n)$ and auxiliary variables $V(n)$ of the AMO encoding of one group of n literals (the whole formula is given by (8)). Commander uses $g = 3$; product recurses down to six literals.

Encoding	$A(n)$	$V(n)$
pairwise	$n(n-1)/2$	0
sequential, PySAT seqcounter	$3n - 4$	$n - 1$
binary, PySAT bitwise	$n\lceil \log_2 n \rceil$	$\lceil \log_2 n \rceil$
commander ($g = 3$)	recursion, about $3.5n$	recursion, about $n/2$
product	recursion, $2n + O(\sqrt{n})$	recursion, $O(\sqrt{n})$
PySAT ladder	$4n$ (fitted)	$n + 1$ (fitted)

3.3 The eight AMO encodings

For a group $L = (\ell_1, \ldots, \ell_m)$ with $m = n$ the encodings are as follows.

Pairwise. Equation (2); no auxiliary variables.

Sequential counter [8]. Auxiliary $s_1, \ldots, s_{m-1}$ with s_i meaning "some ℓ_j, $j \le i$, is true": clauses $(\neg\ell_1 \vee s_1)$; for $1 < i < m$: $(\neg\ell_i \vee s_i)$, $(\neg s_{i-1} \vee s_i)$, $(\neg\ell_i \vee \neg s_{i-1})$; and $(\neg\ell_m \vee \neg s_{m-1})$.

Binary [10]. With $k = \lceil \log_2 m \rceil$ auxiliary bits $b_0, \ldots, b_{k-1}$, literal ℓ_i implies the bit pattern of its index i: clauses $(\neg\ell_i \vee b_j^{(i)})$ for all $j < k$, where $b_j^{(i)}$ is b_j if bit j of i is 1 and $\neg b_j$ otherwise. Two true literals would force one bit both true and false.

Commander [9]. Split L into groups of at most $g = 3$ literals. A group G with two or more literals gets a commander variable c_G with pairwise AMO inside G, the clause $(\neg c_G \vee \bigvee_{\ell \in G} \ell)$ and clauses $(\neg\ell \vee c_G)$ for $\ell \in G$; a singleton group is its own commander. The commanders are constrained by the same encoding recursively, down to at most three literals, which are encoded pairwise.

Product [11]. Place the m literals on a $p \times q$ grid with $p = \lceil \sqrt{m} \rceil$, $q = \lceil m/p \rceil$, add one auxiliary variable per row (u_i) and per column (w_j) and the clauses $(\neg\ell \vee u_i)$, $(\neg\ell \vee w_j)$ for the literal ℓ at (i, j), then apply the encoding recursively to the row variables and to the column variables (plain pairwise at six literals or fewer). Two distinct literals differ in their row or column, so both could not be true.

PySAT seqcounter, bitwise, ladder [5]. The same families generated by the library routine `CardEnc.atmost` with bound 1; the ladder encoding goes back to Gent and Prosser [12]. They are included to compare my implementations with the library's.

3.4 Size of the encodings

Table 1 gives $A(n)$ and $V(n)$ for a group of n literals. For pairwise, sequential and binary they follow directly from the definitions; for commander and product they are given by the recursions above and evaluated numerically. For the three PySAT encodings, seqcounter and bitwise coincide with my sequential and binary encodings in every measured size, and the ladder formula was fitted to the generated CNFs (I did not derive it from the library source). Substituting into (8) and the clue count c of the actual puzzle reproduced the number of clauses and variables of all 47 measured encodings exactly (Section 6.2). In particular the pairwise encoding needs $4n^2 \cdot n(n-1)/2 = 2n^3(n-1) = \Theta(n^4)$ clauses, against $\Theta(n^3)$ for sequential, commander and product, and $\Theta(n^3 \log n)$ for binary.

4 Implementation

The solver is written in Python and uses PySAT. The main modules are: `encodings/alo.py` and `encodings/amo.py` (ALO and the eight AMO encoders, registered in `AMO_ENCODERS`), `sudoku_encoder.py` (CNF generation and model decoding), `solver.py` (solving and uniqueness check), `puzzle_io.py` (text format, generation, solution checker), `baselines/` (CP-SAT, Gurobi, CPLEX MIP, CPLEX CP, CBC), `benchmark.py` and `plot_results.py`. The command line solves a puzzle file with a chosen encoding and solver (`--amo`, `--solver`, `--box`, `--no-unit-alo`, `--check-unique`).

Input validation and decoding. Clues are validated before encoding (type, range, conflicting clues on one cell). A SAT model is decoded cell by cell and the resulting grid is re-validated against the Sudoku rules; a model with no or more than one value in a cell, or with a rule violation, raises an error instead of returning a wrong grid.

Uniqueness check. After a first solution, a blocking clause over the n^3 cell variables is added to the same incremental solver and the formula is solved again. An UNSAT answer proves that the solution is unique; otherwise the second solution is reported. Auxiliary variables are deliberately not blocked.

Puzzle generation. A solved grid is built from the classical base pattern $(rb + \lfloor r/b \rfloor + c) \bmod n$, then rows are shuffled within bands, bands are shuffled, columns are shuffled within stacks, stacks are shuffled, and symbols are permuted, all with a local random generator seeded per puzzle (seed $= 1000\,b +$ puzzle index). Each cell is then kept as a clue independently with probability 0.4. This *puncturing* guarantees that the generated completion is a solution but *not* that it is the only one, so the puzzles can have several solutions. That is acceptable for measuring solve time but not for producing puzzles with a unique solution.

Benchmark harness. Each job (one encoding, solver, puzzle) runs in its own forked process, so a hard instance can be terminated at the time limit and memory is released between jobs. Rows are appended to the CSV as soon as they finish, and a restart skips jobs already present. A `meta.json` file records, for every invocation, the command, parameters, Python and library versions, CPU count, platform, which PySAT solvers were usable, and a content hash and seed for every puzzle; puzzles up to 25×25 are also saved as text files. Every solution returned by a solver is re-checked by an independent routine that does not use the encoder, and the result is stored in the `valid` column. The script `plot_results.py --audit` checks the CSV schema, duplicate job keys, status values, timing consistency, that every `solved` row has `valid=True`, and that the saved puzzles match their recorded hashes and regenerate from their seeds. The repository also contains unit tests (exhaustive AMO tests, end-to-end solving with every encoding, uniqueness, input validation, baselines) and a continuous-integration workflow.

5 Experimental setup

Environment. All numbers in Section 6 come from one machine. The recorded environment (`results/meta.json`) is Python 3.14.4 on Linux 7.0.0-38-generic (x86-64, glibc 2.43) with 12 logical CPUs (the CPU model is not recorded), python-sat 1.9.dev16, OR-Tools 9.15.6755, Gurobi (gurobipy) 13.0.3, CPLEX 22.2.0.1 with docplex 2.32.264, PuLP 2.9.0 (CBC), pandas 3.0.6 and matplotlib 3.11.2. The SAT solvers are used through their PySAT names `cadical195`, `glucose42`, `kissat404` and `minisat22`.

Table 2: Number of runs per experiment and final status. `license_limit`: the free edition of the solver refused the model because of its size limit.

Experiment	encoded	license_limit	solved	timeout
ablation	0	0	29	7
baseline	0	23	32	5
sat	0	0	383	1
size	47	0	0	0

Instances. Box sizes $b = 3, 4, 5, 6$ ($n = 9, 16, 25, 36$), three puzzles per size, generated and punctured as described in Section 4 with keep ratio 0.4. The number of distinct clues per puzzle is 32, 34, 40 ($n = 9$); 120, 103, 101 ($n = 16$); 249, 262, 247 ($n = 25$); and 498, 489, 525 ($n = 36$). The uniqueness of the solution of these puzzles was not checked.

Experiments. The four experiments and their numbers of runs are:

- *sat*: 8 encodings × 4 solvers × 4 sizes × 3 puzzles = 384 runs;
- *ablation*: pairwise, sequential and product without the unit ALO clauses, CaDiCaL only, $3 \times 4 \times 3 = 36$ runs;
- *baseline*: CP-SAT, Gurobi, CPLEX MIP, CPLEX CP and CBC, $5 \times 4 \times 3 = 60$ runs;
- *size*: encode-only runs (no solving) of the eight encodings for $n = 9, \ldots, 64$ on the first puzzle of each size, 47 runs (the pairwise encoding of 64×64 was skipped on purpose, see Section 6.2).

Table 2 summarises the outcome of all 527 recorded runs.

Metrics. For SAT runs, t_{encode} is the time to build the CNF in Python and t_{solve} the time to load the clauses into the solver and solve; t_{total} is their sum. For the baselines, t_{encode} is model building and t_{solve} the solver call. The time limit is 60 s per run; a run that hits it is recorded as `timeout` and is plotted and ranked at 60 s (the raw CSV rows of timed-out runs show 65 s for SAT and 90 s for baselines, because the harness adds a safety margin before killing the process). Tables and figures report the median over the three puzzles of a configuration (or over the 12 runs of an encoding when the four solvers are pooled), together with the minimum and maximum where stated. Each configuration was run once. All SAT solvers run on one thread, and the baselines are pinned to one thread as well (CP-SAT `num_workers` = 1, Gurobi `Threads` = 1, CPLEX `Workers` = 1 or `threads` = 1, CBC `threads` = 1).

Run history. The results CSV combines the output of five invocations of the benchmark script on the same day, recorded in `meta.json`. The baselines for Gurobi, CPLEX and CBC were initially recorded as `missing_library` because the packages were not installed; those rows contained no measurements and were removed before the missing jobs were run after installing the packages. The encode-only experiment was added in a last invocation. `meta.json` records two different repository commits across these invocations, and rows do not carry a commit identifier.

6 Results and discussion

6.1 Correctness of all runs

All 444 runs with status `solved` passed the independent solution check (`valid=True`), and the audit script passes on the final CSV.

Table 3: Number of clauses of the CNF for each AMO encoding (including ALO and clue clauses) for the first puzzle of each size. The pairwise entry at $n = 64$ is empty because that encoding was not generated.

Encoding	$n = 9$	$n = 16$	$n = 25$	$n = 36$	$n = 49$	$n = 64$
pairwise	12 020	124 024	752 749	3 271 602	11 304 899	–
sequential	7 808	46 200	180 249	544 818	1 383 967	3 098 293
commander	8 132	52 344	202 749	627 762	1 604 859	3 573 429
binary	12 020	66 680	315 249	1 125 426	2 834 171	6 309 557
product	8 132	46 200	177 749	534 450	1 335 947	2 836 149
PySAT seqcounter	7 808	46 200	180 249	544 818	1 383 967	3 098 293
PySAT bitwise	12 020	66 680	315 249	1 125 426	2 834 171	6 309 557
PySAT ladder	12 020	66 680	252 749	752 178	1 892 979	4 212 405

Table 4: Number of variables (cell variables n^3 plus auxiliary variables) for the same CNFs.

Encoding	$n = 9$	$n = 16$	$n = 25$	$n = 36$	$n = 49$	$n = 64$
pairwise	729	4 096	15 625	46 656	117 649	–
sequential	3 321	19 456	75 625	228 096	578 641	1 294 336
commander	1 701	11 264	43 125	134 784	348 145	770 048
binary	2 025	8 192	28 125	77 760	175 273	360 448
product	2 673	12 288	40 625	108 864	367 353	720 896
PySAT seqcounter	3 321	19 456	75 625	228 096	578 641	1 294 336
PySAT bitwise	2 025	8 192	28 125	77 760	175 273	360 448
PySAT ladder	3 969	21 504	80 625	238 464	597 849	1 327 104

6.2 Encoding sizes

Table 3 and Table 4 give the number of clauses and variables of the generated CNF (first puzzle of each size) for all eight encodings, and Figure 1 plots them together with the encoding time. The numbers agree exactly with (8) and Table 1. The differences are large: at $n = 49$, the pairwise encoding has 11.3 million clauses, about 8.5 times the 1.34 million of the product encoding, but only 118 thousand variables against 367 thousand for product (the pairwise encoding adds no auxiliary variables). At $n = 36$ the clause ratio between pairwise and product is about 6.1. The pairwise encoding of the 64×64 grid would have $2n^3(n-1) \approx 33$ million group clauses (before ALO and clue clauses), which did not fit in memory as Python lists, so that run was skipped and the corresponding entry is empty. The sequential encoding and its PySAT counterpart are identical in clause and variable counts at all sizes, and so are the binary and PySAT bitwise encodings; for $n \geq 25$ the PySAT ladder encoding has more clauses than sequential and fewer than binary.

6.3 Solving time of the SAT encodings

Figure 2 shows the median total time per encoding for CaDiCaL, and Table 5 lists the medians for all solvers at $n = 25$ and $n = 36$.

Small grids. For $n = 9$ all 32 (encoding, solver) medians lie between 0.006 s and 0.013 s. For $n = 16$ they lie between 0.023 s and 0.059 s, and here the order of the encodings is nearly the same for all four solvers (Spearman rank correlation between solvers from 0.95 to 1.00): pairwise has the highest median time under all four solvers, product (CaDiCaL, Glucose, Kissat) or sequential (MiniSat) the lowest. The absolute gaps are a few tens of milliseconds. Since t_{solve} includes loading the clauses into the solver, the larger CNF of the pairwise encoding (124 thousand clauses against 46 thousand for sequential and product) is a plausible contributor; I did not test this explanation separately.

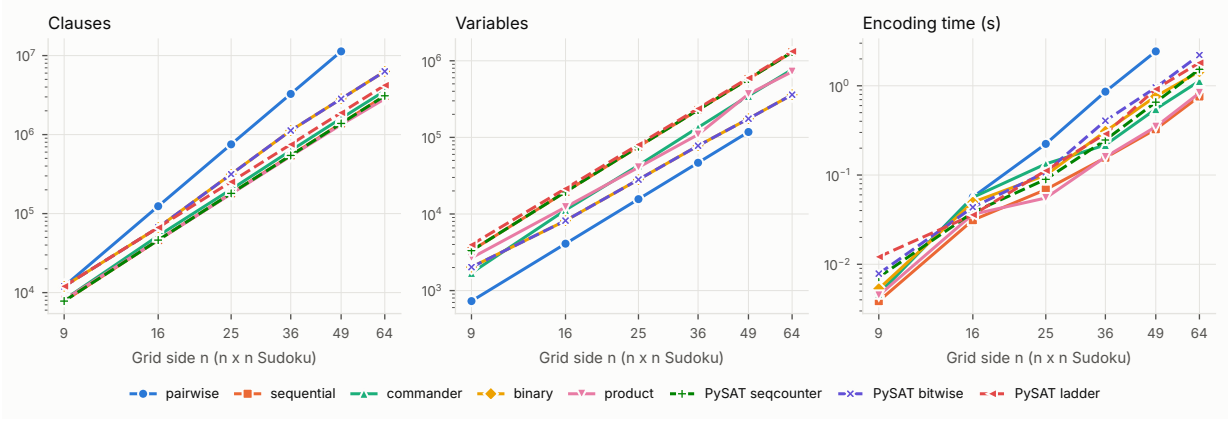


Figure 1: CNF size and encoding time of the eight AMO encodings against grid side (logarithmic axes); one puzzle per size. The pairwise curve ends at $n = 49$.

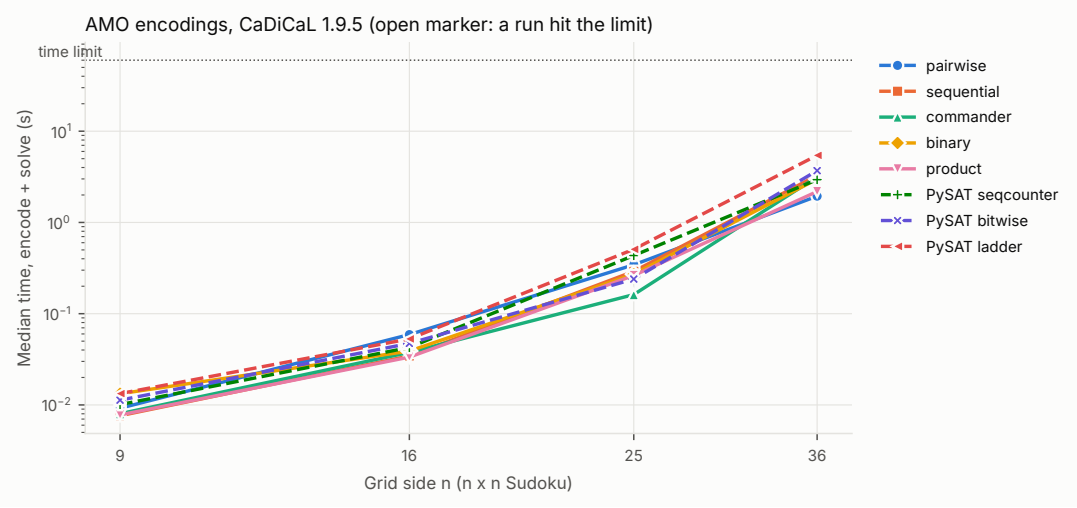


Figure 2: Median total time (encode and solve) of each encoding with CaDiCaL, over three puzzles; log axes. Open markers would indicate a configuration with a timeout (none occurs for CaDiCaL in this experiment).

Medium grids. For $n = 25$ the medians range from 0.10 s to 0.51 s. The pairwise encoding has the highest median under Glucose, Kissat and MiniSat; under CaDiCaL the PySAT ladder encoding is slowest and pairwise is sixth of eight. The best encoding is not the same for every solver (commander for CaDiCaL, product for Glucose, sequential for Kissat and MiniSat), and the rank correlations between solvers fall to 0.38–0.74.

Large grids. At $n = 36$ (Table 5 and Figure 3) the picture changes. The best encoding by median is pairwise for CaDiCaL (1.94 s) and MiniSat (1.53 s), PySAT bitwise for Glucose (1.82 s) and binary for Kissat (0.92 s). The pairwise encoding is thus first under two solvers, second under Glucose, and last under Kissat (2.37 s). The rank correlations between solvers now range from -0.55 to 0.62, i.e. the ordering of the encodings is not stable across solvers. The slowest encoding is the PySAT ladder encoding under CaDiCaL and Glucose (5.43 s and 25.4 s) and sequential under MiniSat (6.14 s), and Glucose needs almost the same time (25.4 s) with the sequential encoding. The only timeout among the 384 SAT runs is binary with Glucose on the third 36×36 puzzle.

To see how much of the variation in time is due to the encoding, the solver, or the puzzle, I computed for each size the share of the variance of $\log_{10} t_{\text{total}}$ explained by each factor alone (between-group sum of squares over total, 96 runs per size). At $n = 16$ the encoding explains 70.5%, the solver 20.7% and the puzzle 1.2%; at $n = 25$ the shares are 16.8%, 28.5% and 11.6%;

Table 5: Median total time in seconds over the three puzzles for every encoding and solver. †: one of the three runs reached the 60 s limit (counted as 60 s).

	$n = 25$				$n = 36$			
Encoding	CaDiCaL	Glucose	Kissat	MiniSat	CaDiCaL	Glucose	Kissat	MiniSat
pairwise	0.34	0.26	0.32	0.25	1.94	2.04	2.37	1.53
sequential	0.29	0.17	0.20	0.12	3.28	25.4	1.15	6.14
commander	0.16	0.14	0.21	0.17	3.16	14.7	1.11	2.55
binary	0.27	0.18	0.22	0.16	2.98	14.7†	0.92	3.55
product	0.26	0.10	0.28	0.12	2.18	9.14	1.31	2.80
PySAT seqcounter	0.43	0.17	0.25	0.19	2.94	23.6	1.78	1.79
PySAT bitwise	0.24	0.18	0.23	0.20	3.70	1.82	1.27	2.11
PySAT ladder	0.51	0.20	0.28	0.20	5.43	25.4	1.40	5.70

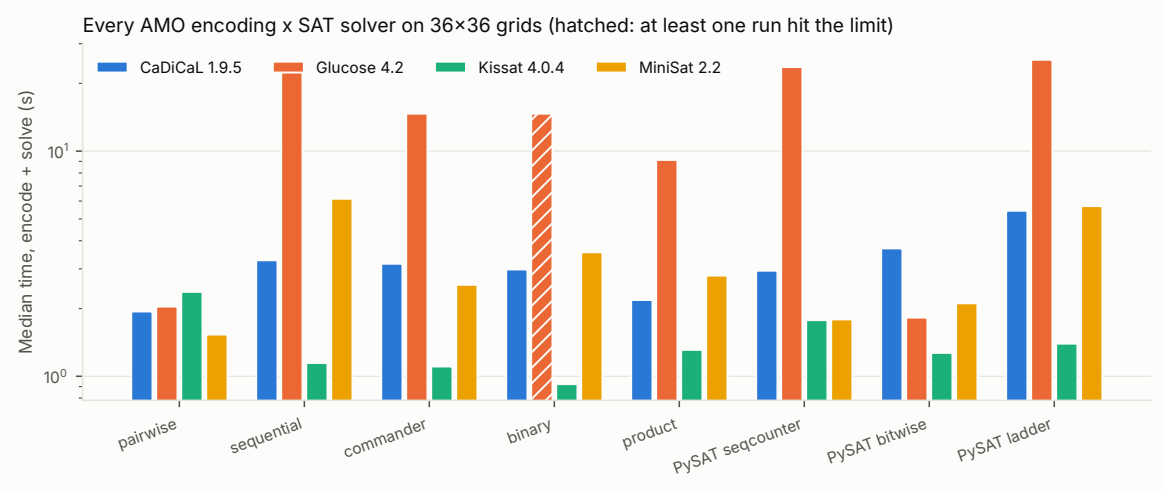


Figure 3: Median total time of every encoding and solver on 36×36 grids (three puzzles; log axis). Hatched bars: at least one of the runs reached the limit.

at $n = 36$ they are 8.1%, 47.5% and 6.4%. These are descriptive effect sizes of main effects, not significance tests, and the rest of the variance is interaction and residual. They are consistent with the observation above: the encoding matters most where runs are shortest, and at $n = 36$ the choice of solver (and its interaction with the encoding) matters more than the choice of encoding.

6.4 Encoding time versus solving time

Table 6 separates the two parts at $n = 36$ for each encoding, pooling the four solvers and three puzzles (12 runs per row). The pairwise encoding has the highest median encoding time (0.55 s against 0.11 s for sequential), the lowest median solve time (1.43 s) and the narrowest range of total times (1.20–4.29 s). The other encodings have median solve times of 1.64–5.32 s and total times that reach 16.8 s (product) up to the limit (binary). Longer encoding time therefore did not translate into longer total time at this size, and the sequential encoding, which is cheapest to build, has one of the slowest median solve times. I did not investigate why pairwise runs are both slower to build and more consistent to solve; its absence of auxiliary variables is one possibility, but this was not tested.

6.5 Ablation: removing the unit ALO clauses

Table 7 and Figure 4 compare CaDiCaL with and without the redundant ALO clauses of rows, columns and boxes (Proposition 1). For $n = 9$ and $n = 16$ the ratio of the median times is between

Table 6: Median encode and solve time and range of the total time (seconds) per encoding at $n = 36$, pooled over the four solvers and three puzzles. > 60: at least one run reached the limit.

Encoding	t_{encode}	t_{solve}	min t_{total}	max t_{total}
pairwise	0.55	1.43	1.20	4.29
sequential	0.11	3.49	0.65	32.8
commander	0.17	2.69	0.77	37.3
binary	0.24	3.02	0.65	>60
product	0.13	2.22	1.04	16.8
PySAT seqcounter	0.21	3.21	0.77	31.5
PySAT bitwise	0.32	1.64	1.01	24.5
PySAT ladder	0.24	5.32	1.38	57.3

Table 7: Median total time (s) of CaDiCaL with and without the unit ALO clauses, their ratio, and the number of timeouts without them. $\geq$: ratio is a lower bound because all three runs hit the limit.

Encoding	n	with unit ALO	without	ratio	timeouts (without)
pairwise	9	0.009	0.011	1.22	0/3
pairwise	16	0.059	0.062	1.06	0/3
pairwise	25	0.34	0.50	1.44	0/3
pairwise	36	1.94	22.2	11.5	1/3
sequential	9	0.008	0.008	1.05	0/3
sequential	16	0.034	0.039	1.13	0/3
sequential	25	0.29	1.03	3.53	0/3
sequential	36	3.28	60.0	$\geq$18.3	3/3
product	9	0.008	0.008	0.99	0/3
product	16	0.033	0.034	1.02	0/3
product	25	0.26	1.56	5.96	0/3
product	36	2.18	60.0	$\geq$27.5	3/3

0.99 and 1.22. For $n = 25$ the ratios are 1.44 (pairwise), 3.53 (sequential) and 5.96 (product), and for $n = 36$ sequential and product time out on all three puzzles and pairwise on one of three (median 22.2 s against 1.94 s). Since timed-out runs are counted as 60 s, the ratios at $n = 36$ for sequential and product are lower bounds. In this setting the redundant clauses therefore clearly helped CaDiCaL on larger grids, although they do not change the set of solutions. The experiment covers one solver and three puzzles per size and cannot say whether the same holds for other solvers.

6.6 Comparison with ILP and CP baselines

Table 8 and Figure 5 compare the SAT approach with the baselines. The "SAT best" column is the lowest median over all 32 (encoding, solver) pairs at each size, which is a favourable selection for the SAT side; I show it to indicate the range of the SAT results and not as a method that could be chosen in advance. CP-SAT solved all 12 puzzles with median times of 0.002, 0.027, 0.092 and 0.80 s, close to or below the best SAT medians (0.006, 0.023, 0.10 and 0.92 s); in this setup I therefore cannot claim that the SAT encodings beat CP-SAT. Gurobi and CPLEX MIP solved the 9×9 puzzles but, as installed from pip, rejected all models from 16×16 on because of the free-edition size limit (status `license_limit`); no timing exists for them beyond $n = 9$, and this reflects the license, not the solver's capability. CPLEX CP solved all three 16×16 puzzles (0.014 s) but only one of three at $n = 25$ (7.68 s), the others being refused, and none at $n = 36$. CBC solved the 9 and 16 puzzles (0.019 s and 0.53 s), one of three at $n = 25$ (28.3 s, the other two

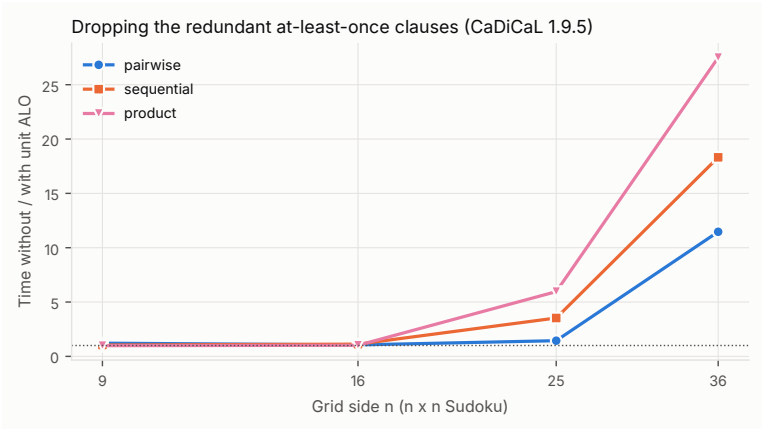


Figure 4: Ratio of the median time without to with the unit ALO clauses (CaDiCaL). Values at $n = 36$ for sequential and product are lower bounds.

Table 8: Median total time (s) of the baselines and of the best SAT configuration. "licence cap": the free edition refused the model. $(k/3)$: number of the three puzzles that were solved, the median being taken over the solved or timed-out runs; > 60: median at the time limit.

n	SAT best	CP-SAT	Gurobi (ILP)	CPLEX MIP	CPLEX CP	CBC (ILP)
9	0.006	0.002	0.006	0.031	0.009	0.019
16	0.023	0.027	licence cap	licence cap	0.014	0.53
25	0.10	0.092	licence cap	licence cap	7.68 (1/3)	>60 (1/3)
36	0.92	0.80	licence cap	licence cap	licence cap	>60 (0/3)

timing out) and none at $n = 36$.

7 Limitations

- *Small sample.* Three puzzles per size and one run per configuration; no statistical tests, and the medians of three runs are sensitive to a single hard puzzle (at $n = 36$ the third puzzle was the hardest by median over all configurations, 4.1 s against 2.1 s and 2.6 s).
- *Puzzle family.* The puzzles are random punctured grids with a 0.4 keep ratio; uniqueness of solutions was not checked, and standard hard benchmark sets (for example 17-clue puzzles) were not used. Results may differ on puzzles with few clues or a unique solution.
- *One machine and software stack.* Timings come from a single machine with one operating system and library versions given in Section 5 (Python 3.14.4); the CPU model was not recorded. Other CPUs and library versions may rank configurations differently.
- *Run history.* The CSV merges five invocations under two repository commits (Section 5); concurrent load on the machine during the runs was not controlled or recorded.
- *Time measure.* t_{solve} includes loading the clauses into the solver, and t_{encode} is measured for Python implementations that differ in implementation effort from the PySAT library routines; differences between a hand-written encoder and its PySAT counterpart (identical CNF) reflect implementation, not encoding.
- *Baselines.* Gurobi and CPLEX MIP could not be evaluated beyond 9×9 and CPLEX CP beyond 25×25 because of free-edition limits, so the comparison with commercial MIP solvers is incomplete. The ILP and CP models are simple direct formulations and were not tuned.
- *Mechanisms.* I report observed differences but did not analyse solver internals (propagation strength, learned clauses), so explanations offered above are hypotheses.
- *Scope.* The grids end at 36×36 for solving; larger sizes were only encoded.

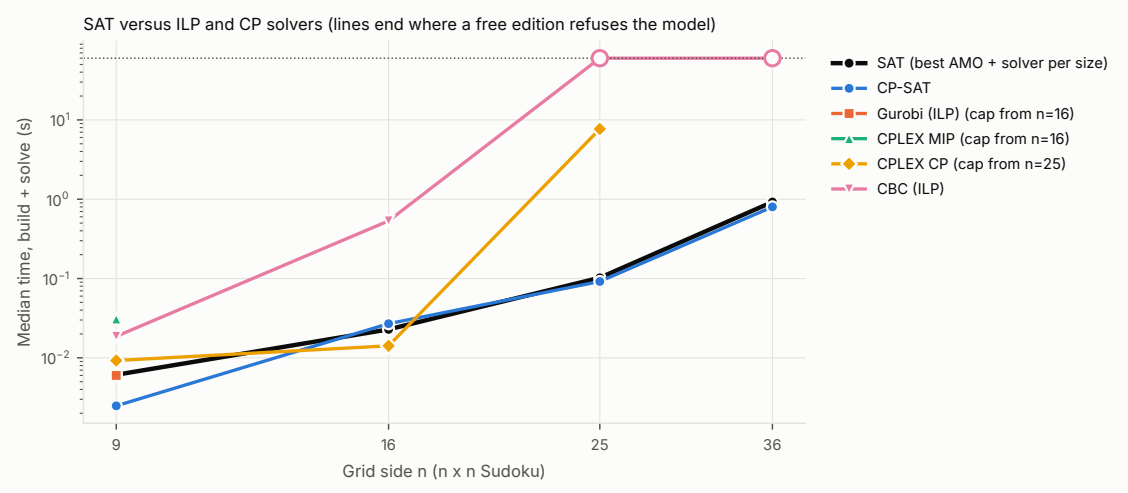


Figure 5: Median build and solve time of the baselines and of the best SAT configuration per size. Open markers: a run reached the limit. Lines end where a free edition refused the model.

8 Conclusion

I encoded Sudoku in CNF with eight AMO encodings and measured both the size of the formulas and the time to solve them. The closed-form clause and variable counts match all generated CNFs, and they show that the pairwise encoding becomes much larger than the others as the grid grows (about 8.5 times the clauses of the product encoding at 49×49). In the solving experiments the differences are small for grids up to 25×25, and at 36×36 no encoding is best for all solvers, the ranking of the encodings changing from solver to solver; the solver explains more of the variation than the encoding there. The redundant per-unit ALO clauses were beneficial for CaDiCaL on the larger grids. CP-SAT was as fast as the best SAT configuration on these puzzles, while the free editions of Gurobi and CPLEX could not process grids from 16×16 on. I therefore do not recommend one AMO encoding over another from these data alone; larger samples, more solvers and harder puzzles would be needed for that. The code, the raw results, the puzzles and the scripts that produced the figures and tables are available from the repository below.

Data availability

The source code, the raw benchmark results (`results/results.csv`), the puzzles used (`results/instances/`), the environment metadata (`results/meta.json`), and the scripts that regenerate the figures and tables are available in the project repository: https://github.com/canknotcawh/sudoku_sat_solver.

References

[1] Takayuki Yato and Takahiro Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E86-A(5):1052–1060, 2003. UNVERIFIED: recalled from memory, not re-checked in this session.

[2] Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In *Theory and Applications of Satisfiability Testing – SAT 2003*, volume 2919 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2004. UNVERIFIED: recalled from memory; the SAT 2003 proceedings appeared as LNCS 2919 in 2004.

[3] Gilles Audemard and Laurent Simon. Predicting learnt clauses quality in modern SAT solvers. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI)*, pages 399–404, 2009. UNVERIFIED: recalled from memory.

[4] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020. In *Proceedings of SAT Competition 2020 – Solver and Benchmark Descriptions*, number B-2020-1 in Department of Computer Science Report Series B, pages 50–53. University of Helsinki, 2020. Recommended citation on the Kissat web page (fmv.jku.at/kissat).

[5] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: A Python toolkit for prototyping with SAT oracles. In *Theory and Applications of Satisfiability Testing – SAT 2018*, volume 10929 of *Lecture Notes in Computer Science*, pages 428–437. Springer, 2018. UNVERIFIED: recalled from memory.

[6] Inês Lynce and Joël Ouaknine. Sudoku as a SAT problem. In *Proceedings of AIMATH 06*, 2006. Checked on the authors' page (J. Ouaknine); the exact venue name should be confirmed.

[7] Tjark Weber. A SAT-based Sudoku solver. In Geoff Sutcliffe and Andrei Voronkov, editors, *LPAR-12, The 12th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning, Short Paper Proceedings*, pages 11–15, 2005. Checked on the author's page.

[8] Carsten Sinz. Towards an optimal CNF encoding of Boolean cardinality constraints. In *Principles and Practice of Constraint Programming – CP 2005*, volume 3709 of *Lecture Notes in Computer Science*, pages 827–831. Springer, 2005. Title and venue found via search (DOI 10.1007/11564751_73); page range from memory.

[9] William Klieber and Gihwon Kwon. Efficient CNF encoding for selecting 1 from N objects. In *Fourth Workshop on Constraints in Formal Verification (CFV)*, 2007. Title and authors found via Semantic Scholar; venue from memory, UNVERIFIED.

[10] Alan M. Frisch and Timothy J. Peugniez. Solving non-Boolean satisfiability problems with stochastic local search. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, 2001. Title and venue found via search; page range omitted.

[11] Jingchao Chen. A new SAT encoding of the at-most-one constraint. In *Proceedings of the 9th International Workshop on Constraint Modelling and Reformulation (ModRef)*, 2010. UNVERIFIED: attribution taken from the source code documentation and memory; no page could be checked.

[12] Ian P. Gent and Patrick Prosser. SAT encodings of the stable marriage problem with ties and incomplete lists. In *Proceedings of the 5th International Symposium on Theory and Applications of Satisfiability Testing (SAT)*, 2002. UNVERIFIED: attribution of the ladder encoding from memory.

[13] Laurent Perron and Vincent Furnon. OR-Tools. Google. URL `https://developers.google.com/optimization`. Software; CP-SAT solver. Accessed October 2026.

[14] Gurobi Optimization, LLC. Gurobi optimizer reference manual. URL `https://www.gurobi.com`. Software. Accessed October 2026.

[15] IBM. IBM ILOG CPLEX Optimization Studio. URL `https://www.ibm.com/products/ilog-cplex-optimization-studio`. Software; includes CPLEX MIP and CP Optimizer. Accessed October 2026.

[16] COIN-OR. CBC (coin-or branch and cut). URL https://github.com/coin-or/Cbc. Software. Accessed October 2026.

[17] Stuart Mitchell, Michael O'Sullivan, and Iain Dunning. PuLP: A linear programming toolkit for Python. The University of Auckland, 2011. UNVERIFIED: recalled from memory.